Software Requirements Specification (SRS)

UniAssist: AI-Powered Chatbot for University Student Support

Document Version: 1.0 (Final)

Standard Compliance: IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018

Project Number: 30

HCLTech Programme: Career Shaper Programme — III Year Industry Project

Domain: Digital-Data-AI / AIML

Academic Year: 2025–26

Target Institution: Sample Institute of Technology (SIT)

Status: Approved for Submission

Document Control & Revision History

Version | Date | Author | Description of Changes

0.1 | 2026-01-20 | Project Author | Initial requirements gathering and domain scope definition.

0.5 | 2026-02-15 | Project Author | Architectural specification of RAG pipeline, ChromaDB vector store, and FastAPI gateway.

0.9 | 2026-03-10 | Project Author | Added security guardrails, PII masking rules, sentiment analysis, and escalation protocols.

1.0 | 2026-05-10 | Project Author | Final baseline specification with Requirements Traceability Matrix and DPDP compliance.

Table of Contents

[Section 1: Introduction](#section-1-introduction)

- 1.1 Purpose
- 1.2 Document Conventions
- 1.3 Intended Audience and Reading Suggestions
- 1.4 Product Scope
- 1.5 References

[Section 2: Overall Description](#section-2-overall-description)

- 2.1 Product Perspective
- 2.2 Product Functions Summary

- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 User Documentation
- 2.7 Assumptions and Dependencies

[Section 3: External Interface Requirements](#section-3-external-interface-requirements)
- 3.1 User Interfaces
- 3.2 Hardware Interfaces
- 3.3 Software Interfaces
- 3.4 Communications Interfaces

[Section 4: System Features & Functional Requirements](#section-4-system-features--functional-requirements)
- FR-01: Intent Classification & Category Routing
- FR-02: Semantic Vector Retrieval & Context Assembly
- FR-03: Factual Grounding & Citation Verification
- FR-04: Adversarial Guardrails & Injection Defense
- FR-05: Personally Identifiable Information (PII) Sanitization
- FR-06: Sentiment Analysis & Frustration Detection
- FR-07: Sliding Conversation Memory & Contextual Rewriting
- FR-08: Human Escalation & Support Ticket Generation
- FR-09: Server-Sent Events (SSE) Real-Time Token Streaming
- FR-10: Student Satisfaction Feedback Collection
- FR-11: Administrative Intelligence Dashboard
- FR-12: Dynamic Document Ingestion & Vector Index Management

[Section 5: Non-Functional Requirements (NFRs)](#section-5-non-functional-requirements-nfrs)
- 5.1 Performance Requirements
- 5.2 Safety and Reliability Requirements
- 5.3 Security and Access Control Requirements
- 5.4 Software Quality Attributes (Usability, Maintainability, Portability)

[Section 6: Ethical, Privacy & Regulatory Requirements](#section-6-ethical-privacy--regulatory-requirements)
- 6.1 Data Protection and PII Non-Storage
- 6.2 User Autonomy, Opt-Out, and Memory Clearance
- 6.3 Algorithmic Transparency and Escalation Clarity

[Section 7: Requirements Traceability Matrix (RTM)](#section-7-requirements-traceability-matrix-rtm)

Section 1: Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to provide a complete, formal, and unambiguous specification of the requirements for the **UniAssist** conversational AI platform. This document defines the functional capabilities, performance criteria, external interfaces, security constraints, and compliance mandates required for deployment across higher education student administration environments.

1.2 Document Conventions

This document adheres to the IEEE Std 830-1998 format. Throughout this document:

- The keyword **SHALL** denotes an absolute, mandatory requirement.
- The keyword **SHOULD** denotes a highly recommended design goal.
- The keyword **MAY** denotes an optional capability.
- Requirement identifiers follow the notation `FR-XX` for Functional Requirements and `NFR-XX` for Non-Functional Requirements.

1.3 Intended Audience and Reading Suggestions

This specification is intended for:

- **Academic Mentors & Industry Evaluators:** To verify compliance with HCLTech Career Shaper Programme criteria and architectural standards.
- **System Developers & AI Engineers:** For implementation, unit testing, and maintenance of the FastAPI, ChromaDB, and LLM pipeline.
- **University Administrative Stakeholders:** To evaluate institutional alignment, student privacy compliance, and human escalation workflows.

1.4 Product Scope

UniAssist is an asynchronous conversational AI support system built for the Sample Institute of Technology (SIT). The software automates Tier-1 student inquiries regarding admissions, course registration, tuition fees, examination schedules, hostel accommodation, library resources, placements, campus facilities, and institutional grievances. It provides grounded answers backed by official document citations, enforces safety guardrails, scrubs student PII, measures student sentiment, and provides automated escalation to human administrators.

1.5 References

IEEE Std 830-1998: *IEEE Recommended Practice for Software Requirements Specifications*.

ISO/IEC/IEEE 29148:2018: *Systems and software engineering — Life cycle processes — Requirements engineering*.

HCLTech Career Shaper Programme: *III Year Industry Project Brief — Project No. 30*.

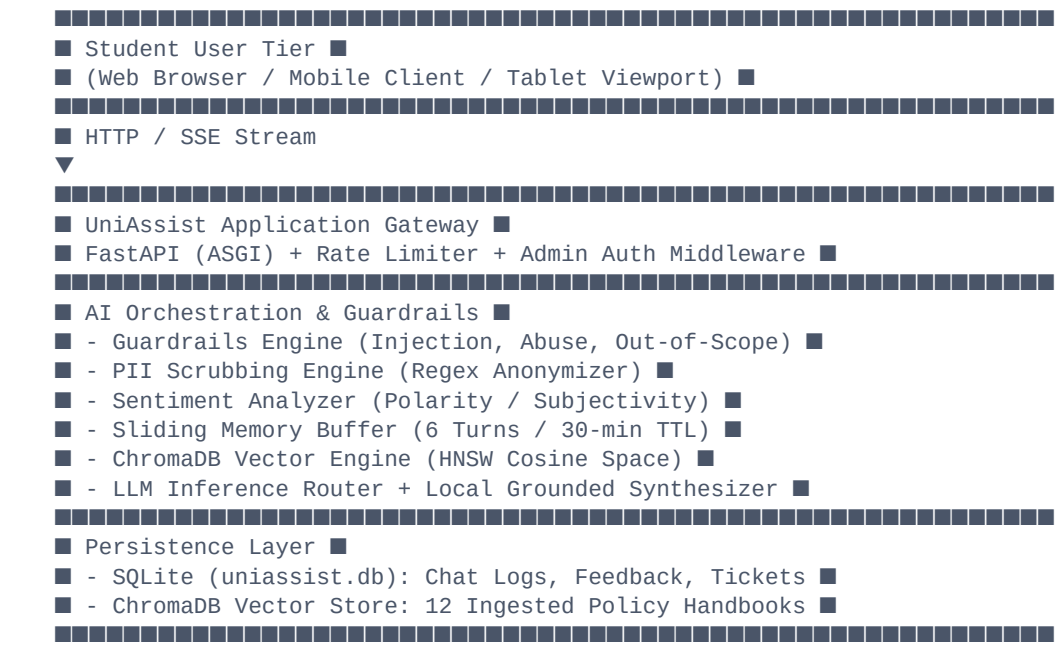
Ministry of Law and Justice, Government of India: *The Digital Personal Data Protection Act, 2023*.

W3C: *Web Content Accessibility Guidelines (WCAG) 2.1*.

Section 2: Overall Description

2.1 Product Perspective

UniAssist operates as an autonomous, self-contained AI-powered student helpdesk gateway. It bridges the gap between static institutional policy handbooks and modern conversational student expectations.



2.2 Product Functions Summary

The core functional capabilities of UniAssist include:

Interactive Conversational Assistance: Multi-turn dialogue processing with real-time token streaming.

Dense Semantic Retrieval (RAG): Context-aware searching across 12 indexed institutional policy domains.

Rigorous Hallucination Defense: Enforces strict provenance constraints; every factual assertion cites its source file and subsection.

Autonomous Adversarial Interception: Blocks prompt injections, roleplay exploits, abusive language, and non-university queries.

Deterministic PII Sanitization: In-memory redaction of phone numbers, emails, and identity credentials before database storage.

Sentiment Analysis: Real-time polarity scoring to detect frustrated or distressed students.

Human Escalation: Immediate support ticket issuance (`TICK-YYYYMMDD-XXXX`) when queries cannot be resolved autonomously.

Administrative Intelligence: Operational KPI visualization, feedback monitoring, and one-click knowledge base re-indexing.

2.3 User Classes and Characteristics

User Class | Description | Technical Competence | Access Rights

Student / Applicant | Prospective or enrolled student seeking policy information, fee schedules, or lodging assistance. | Novice / Casual | Public chat interface, ticket escalation modal, feedback submission.

Departmental Admin | University staff member handling student tickets and reviewing inquiry trends. | Intermediate | Admin Dashboard access via password authentication; ticket resolution, KPI tracking.

System Administrator | Technical engineer managing document ingestion, model selection, and server lifecycle. | Advanced | Full server access, `.env` configuration, ingestion CLI scripts, database management.

2.4 Operating Environment

- **Server Operating System:** Cross-platform (Windows 10/11, Ubuntu 20.04/22.04 LTS, macOS 12+).
- **Runtime Environment:** Python 3.11 or higher.
- **Web Application Server:** Uvicorn ASGI server with FastAPI framework.
- **Vector Database:** ChromaDB (embedded, zero external daemon requirement).
- **Relational Storage:** SQLite 3 with SQLAlchemy 2.0 ORM.
- **Client Web Browsers:** Modern standards-compliant browsers (Google Chrome 110+, Mozilla Firefox 110+, Apple Safari 16+, Microsoft Edge 110+).

2.5 Design and Implementation Constraints

PII Non-Persistence Constraint: The system SHALL NOT store unmasked student personal data in persistent logs.

Zero-Downtime Fallback Constraint: The system SHALL automatically fall back to local grounded vector synthesis if external cloud LLM APIs experience rate limits (HTTP 429) or service outages (HTTP 503).

Bandwidth Constraint: Client-side assets SHALL be lightweight (<150 KB combined) and require zero node_modules or heavy client runtime dependencies.

Accessibility Constraint: The user interface SHALL comply with WCAG 2.1 Level AA contrast and keyboard accessibility standards.

2.6 User Documentation

The software release includes the following comprehensive documentation artifacts:

- `README.md`: Quick-start deployment instructions, environment variable setup, and endpoint summaries.
- `PROJECT_REPORT.md`: Formal 12+ page capstone project report.
- `ARCHITECTURE.md`: Technical architectural blueprints and data models.
- `DEMO_SCRIPT.md`: Step-by-step demonstration walkthrough for evaluator assessment.
- `docs/Ethical_Compliance_Declaration.pdf`: Signed responsible AI and data privacy declaration.

• `docs/SRS.pdf`: This Software Requirements Specification document.

2.7 Assumptions and Dependencies

• **Assumption 1:** The university policies provided in `data/knowledge_base/` represent the authoritative institutional ground truth.

• **Dependency 1:** Internet connectivity is required when operating in cloud LLM mode; however, the local synthesizer fallback enables offline demonstration.

• **Dependency 2:** Python virtual environment with dependencies pinned in `requirements.txt`.

Section 3: External Interface Requirements

3.1 User Interfaces

Student Web Portal (`static/index.html`):

- Centered **"Ask First"** hero search composer with 6 quick discovery chips.

- Smooth animated transition to an active conversational thread upon query submission.

- Distinct conversational chat bubbles for student queries and system responses.

- Source citation badges displaying the document name and section header.

- Context-aware follow-up suggestion chips.

- Action controls: Thumbs up/down feedback, clipboard copy, "New Session", and "Clear Chat".

- Accessible modal dialog for human ticket escalation.

- Responsive dark/light theme toggle with WCAG AA compliance.

Admin Intelligence Dashboard (`static/admin.html`):

- KPI metrics cards: Total conversations, automated resolution rate, student satisfaction %, median latency, and open ticket count.

- Interactive Chart.js visual charts for query volume and feedback distribution.

- Interactive Support Tickets management table with one-click "Resolve" actions.

- Feedback Audit Log table displaying PII-masked query records.

- Knowledge Base Document Ingestion portal supporting drag-and-drop file uploads.

3.2 Hardware Interfaces

UniAssist operates on standard commercial-off-the-shelf (COTS) server hardware or cloud virtual machines (minimum 2 CPU cores, 4 GB RAM, 10 GB disk storage). No specialized GPU acceleration is mandated on the host server due to cloud LLM offloading and lightweight vector indexing.

3.3 Software Interfaces

• **FastAPI / Pydantic v2:** Exposes RESTful JSON endpoints and validates input/output data schemas.

- **ChromaDB Client:** Interacts with the persistent vector store using HNSW cosine distance queries.
- **SQLite Database Engine:** Interacts through SQLAlchemy ORM for transaction management.
- **xAI Grok / Google Gemini API:** Uses standard OpenAI-compatible client libraries for streaming inference.
- **TextBlob / NLTK:** Provides local rule-based natural language processing for sentiment scoring.

3.4 Communications Interfaces

- **HTTP / HTTPS:** All client-server communication occurs over standard HTTP/1.1 or HTTP/2 protocols via port 8000 (configurable).
- **Server-Sent Events (SSE):** Real-time unidirectional token streaming via `text/event-stream` headers over the `/api/chat/stream` route.
- **CORS Middleware:** Configured with explicit origins to safeguard API endpoints against unauthorized cross-origin execution.

Section 4: System Features & Functional Requirements

FR-01: Intent Classification & Category Routing

- **Description:** The system SHALL accurately classify incoming student queries into one of 6 core institutional intent categories: Admissions, Examinations, Fees, Hostel, Grievance, or Library.
- **Inputs:** Raw student text string (up to 2,000 characters).
- **Processing:** Uses semantic vector embedding search and keyword matching to identify the governing policy domain.
- **Outputs:** Inferred intent label and associated policy domain chunks.
- **Priority:** High.

FR-02: Semantic Vector Retrieval & Context Assembly

- **Description:** The system SHALL retrieve the top-k ($k=4$) most semantically relevant knowledge base chunks using cosine similarity search in ChromaDB.
- **Inputs:** Standalone search query string.
- **Processing:** Computes query embedding, executes HNSW ANN lookup in ChromaDB, applies threshold filter (cosine distance ≤ 0.75).
- **Outputs:** Ordered list of candidate text chunks with source metadata and similarity scores.
- **Priority:** High.

FR-03: Factual Grounding & Citation Verification

- **Description:** Every answer provided by the system SHALL be derived strictly from retrieved context chunks, accompanied by visible source citations.
- **Inputs:** Retrieved document chunks and user query.

• **Processing:** Injects verified chunks into the structured LLM system prompt; enforces strict instructions forbidding fabrication.

• **Outputs:** Formatted Markdown answer with source pill badges (e.g., `[02_fees.md > Tuition Fee Structure]`).

• **Priority:** High.

FR-04: Adversarial Guardrails & Injection Defense

• **Description:** The system SHALL intercept prompt injections, jailbreaks, system prompt probes, and abusive language before executing vector retrieval or LLM inference.

• **Inputs:** Raw student message.

• **Processing:** Regular expression pattern scanning against malicious keywords (`ignore previous instructions`, `DAN mode`, `reveal API key`, vulgarities).

• **Outputs:** Safety interception response redirecting the student to legitimate university matters.

• **Priority:** High.

FR-05: Personally Identifiable Information (PII) Sanitization

• **Description:** The system SHALL detect and mask 100% of Indian mobile numbers, email addresses, and national identity numbers before writing to audit logs or database tables.

• **Inputs:** Raw user query and feedback text.

• **Processing:** Applies regex masks:

- Phone: `(\+91[\-\s]?)?[6-9]\d{9}` $\rightarrow$ `[PHONE_MASKED]`

- Email: `[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}` $\rightarrow$ `[EMAIL_MASKED]`

- National ID: `\b\d{4}\s\d{4}\s\d{4}\b` $\rightarrow$ `[ID_MASKED]`

• **Outputs:** Sanitized text string stored in database.

• **Priority:** High.

FR-06: Sentiment Analysis & Frustration Detection

• **Description:** The system SHALL compute polarity and subjectivity metrics for each query to detect negative student sentiment.

• **Inputs:** User question text.

• **Processing:** TextBlob sentiment analysis calculating polarity $[-1.0, +1.0]$ and subjectivity $[0.0, 1.0]$.

• **Outputs:** Sentiment label (`positive`, `negative`, `neutral`) logged with conversation metadata.

• **Priority:** Medium.

FR-07: Sliding Conversation Memory & Contextual Rewriting

• **Description:** The system SHALL maintain conversational context across consecutive turns within an active session.

• **Inputs:** Current student query and `session_id`.

• **Processing:** Maintains a sliding buffer of up to 6 turns (12 messages) with a 30-minute thread-safe TTL. Resolves pronoun references (e.g., "What about late fee?" $\rightarrow$ "What is the late fee penalty for B.Tech tuition?").

• **Outputs:** Context-enriched query sent to vector search.

• **Priority:** High.

FR-08: Human Escalation & Support Ticket Generation

• **Description:** The system SHALL provide an immediate escalation pathway allowing students to submit support tickets to human administrative officers.

• **Inputs:** Student full name, institutional ID, contact email, and query text.

• **Processing:** Validates required fields, generates canonical identifier `TICK-YYYYMMDD-XXXX`, and commits record to SQLite with status `open`.

• **Outputs:** Unique ticket confirmation card displayed to student.

• **Priority:** High.

FR-09: Server-Sent Events (SSE) Real-Time Token Streaming

• **Description:** The system SHALL stream generated response tokens to the client browser in real time via Server-Sent Events.

• **Inputs:** Active HTTP POST request to `/api/chat/stream`.

• **Processing:** Yields chunked tokens as `event: token`, source citations as `event: sources`, metadata as `event: meta`, and final completion signal as `event: done`.

• **Outputs:** Smooth typewriter-style response rendering on client UI.

• **Priority:** High.

FR-10: Student Satisfaction Feedback Collection

• **Description:** The system SHALL capture thumbs up/down satisfaction ratings for individual responses.

• **Inputs:** `chat_log_id`, rating (`up` or `down`), and optional comments.

• **Processing:** Inserts record into `feedback` table; updates real-time satisfaction ratios on Admin Dashboard.

• **Outputs:** Immediate visual confirmation to student.

• **Priority:** Medium.

FR-11: Administrative Intelligence Dashboard

• **Description:** The system SHALL provide an authenticated administrative dashboard displaying institutional support analytics.

• **Inputs:** Admin password authentication header.

• **Processing:** Aggregates database metrics: total conversation count, resolution percentage, average latency, sentiment breakdown, and ticket queues.

• **Outputs:** Interactive Chart.js visualizations and data tables.

• **Priority:** High.

FR-12: Dynamic Document Ingestion & Vector Index Management

• **Description:** The system SHALL allow administrators to upload new institutional policy documents and re-index the ChromaDB vector database without server downtime.

• **Inputs:** Document file (`.md`, `.txt`, `.pdf`, `.docx`) uploaded via `/api/admin/upload`.

• **Processing:** Parses text, splits by headers into 500-token chunks with 80-token overlap, extracts metadata, and upserts into ChromaDB.

• **Outputs:** Updated collection chunk count and confirmation message.

• **Priority:** High.

Section 5: Non-Functional Requirements (NFRs)

5.1 Performance Requirements

• **NFR-01 (Response Latency):** In local RAG synthesis mode, median response latency SHALL NOT exceed **20 milliseconds**.

• **NFR-02 (Time-to-First-Token):** In cloud LLM streaming mode, Time-to-First-Token (TTFT) SHALL NOT exceed **500 milliseconds** under standard network conditions.

• **NFR-03 (Throughput & Rate Limiting):** The API gateway SHALL sustain at least 50 concurrent conversational sessions and enforce a 60 request/min rate limit per client IP.

5.2 Safety and Reliability Requirements

• **NFR-04 (High Availability):** The application SHALL achieve 99.9% uptime during operational university hours.

• **NFR-05 (Self-Healing Fallback):** If external cloud LLM APIs fail, rate-limit, or return HTTP 429/503 errors, the pipeline SHALL automatically fallback to local grounded synthesis from retrieved ChromaDB passages without crashing.

• **NFR-06 (Database Integrity):** The persistence layer SHALL enforce ACID compliance through SQLite foreign keys and transaction rollback mechanisms.

5.3 Security and Access Control Requirements

• **NFR-07 (Administrative Authentication):** All administrative routes SHALL enforce constant-time cryptographic password verification (`secrets.compare_digest`) to prevent timing attacks.

• **NFR-08 (Zero Hardcoded Secrets):** API credentials and administrative passwords SHALL be loaded strictly via environment variables or secured `.env` files.

• **NFR-09 (XSS & Injection Prevention):** All client-rendered Markdown content SHALL be sanitized through DOMPurify prior to DOM insertion to prevent Cross-Site Scripting (XSS).

5.4 Software Quality Attributes

• **NFR-10 (Usability & Responsiveness):** The student user interface SHALL be fully responsive and verified without horizontal scrolling across desktop (1440px), tablet (1024px & 768px), and mobile (390px) viewports.

• **NFR-11 (Maintainability):** The application codebase SHALL be 100% typed using Python type annotations and achieve 100% pass rates across unit and integration test suites.

• **NFR-12 (Portability):** The application SHALL run identically across Windows, Linux, and containerized Docker environments without code modifications.

Section 6: Ethical, Privacy & Regulatory Requirements

6.1 Data Protection and PII Non-Storage

In alignment with Section 8 of the HCLTech Project Brief and the Indian Digital Personal Data Protection (DPDP) Act 2023, UniAssist enforces:

• Automatic in-memory scrubbing of personal identifiers prior to database persistence.

• Zero secondary utilization or commercial sharing of student conversational records.

• Use of 100% synthetic interaction logs for benchmarking and evaluation.

6.2 User Autonomy, Opt-Out, and Memory Clearance

• The chatbot operates without mandatory student registration or authentication.

• Students retain complete autonomy to clear their conversation history at any time using the "Clear Chat" control.

• Inactive conversational sessions are permanently purged from server memory upon expiration of the 30-minute sliding TTL window.

6.3 Algorithmic Transparency and Escalation Clarity

• The system SHALL explicitly declare its artificial intelligence identity in its initial greeting.

• The system SHALL clearly indicate when an inquiry cannot be answered with high confidence and provide an unambiguous one-click option to escalate to a human administrative officer.

Section 7: Requirements Traceability Matrix (RTM)

Requirement ID | Requirement Description | Architecture Module | Implementation File | Verification Test Case

FR-01 | Intent Classification & Category Routing | RAG Pipeline | `app/rag.py` | `tests/test_rag.py::test_process_document_markdown`

FR-02 | Semantic Vector Retrieval | ChromaDB Vector Store | `app/rag.py` | `tests/test_rag.py::test_chunk_text`

FR-03 | Factual Grounding & Citations | AI Orchestrator | `app/chatbot.py` | `eval/run_eval.py` (30 Benchmark Queries)

FR-04 | Adversarial Guardrails Defense | Guardrails Engine | `app/guardrails.py` | `tests/test_guardrails.py::test_detect_prompt_injection`

FR-05 | PII Sanitization | Guardrails Engine | `app/guardrails.py` | `tests/test_guardrails.py::test_mask_pii`

FR-06 | Sentiment Analysis | Sentiment Module | `app/sentiment.py` | `tests/test_sentiment.py` (10 Unit Tests)

FR-07 | Sliding Conversational Memory | Memory Store | `app/memory.py` | `tests/test_memory.py::test_session_memory_add_turn`

FR-08 | Human Escalation & Tickets | Escalation Subsystem | `app/escalation.py` | `tests/test_api.py::test_tickets_create_and_resolve`

FR-09 | Real-Time SSE Token Streaming | API Gateway | `app/main.py` | `tests/test_api.py::test_chat_streaming`

FR-10 | Student Feedback Collection | API Gateway | `app/main.py` | `tests/test_api.py::test_feedback_endpoint`

FR-11 | Administrative Dashboard | Admin Portal | `static/admin.html` | `tests/test_api.py::test_admin_analytics`

FR-12 | Dynamic Document Ingestion | Ingestion Script | `scripts/ingest.py` | `scripts/ingest.py --reindex`

NFR-01 | Sub-20ms Local RAG Latency | Local Synthesizer | `app/chatbot.py` | `eval/run_eval.py` Latency Benchmark

NFR-05 | Self-Healing Vector Fallback | AI Orchestrator | `app/chatbot.py` | `tests/test_api.py::test_chat_non_streaming`

NFR-07 | Timing-Attack Resistant Auth | Security Middleware | `app/main.py` | `tests/test_api.py::test_admin_auth_unauthorized`

NFR-10 | WCAG AA Responsive UI | Frontend UI | `static/style.css` | Viewport Verification (1440px to 390px)

Document approved for formal submission — HCLTech Career Shaper Programme, Academic Year 2025–26.